



Python na prática: variáveis, tipos e coleções

Coding: Linguagens e Técnicas · Aula 3 · 25 de agosto de 2026

Prof. Sebastião Rogério

Agenda

01

Variáveis

Atribuição múltipla, troca de valores, tipagem dinâmica e palavras reservadas.

02

Tipos e conversão

input, int, float, str e o problema do dinheiro em ponto flutuante.

03

Condicionais

if, elif, else, valores falsy e a pegadinha do or.

04

Listas, tuplas e dicionários

Índices, fatiamento, métodos, cópia e o efeito colateral que derruba sistema.



Hoje o slide é só o pano de fundo. O que vale é o que aparece no seu terminal.

Abra o VS Code ou o terminal antes de continuar. Python 3.12 ou superior.

01

Variáveis

Como Python guarda um valor e o que isso tem de diferente de Java ou C.

Atribuição, uma por linha

Não existe declaração de tipo. O tipo vem do valor que você atribuiu.

```
variaveis.py

nome = "Ana"
idade = 21
media = 8.75
matriculada = True

print(nome, idade, media, matriculada)
```

SAÍDA

Ana 21 8.75 True

Dá para declarar três variáveis de tipos diferentes na mesma linha? Este código funciona?

```
python  
  
nome, idade, matriculada = "Ana", 21, True  
  
print(nome)  
print(idade)  
print(matriculada)
```

Antes de rodar, escreva no caderno o que você acha que sai.

Funciona. Python desempacota a sequência da direita na ordem da esquerda.

```
python  
  
nome, idade, matriculada = "Ana", 21, True  
  
print(nome)  
print(idade)  
print(matriculada)
```

SAÍDA

Ana

21

True

O lado direito monta uma tupla e o lado esquerdo consome item por item. Os tipos não precisam ser iguais.

Um valor para várias variáveis

Útil para zerar contadores no começo de um programa.

```
variaveis.py  
  
faltas = atrasos = trabalhos = 0  
  
print(faltas, atrasos, trabalhos)
```

SAÍDA

0 0 0

E se a quantidade de nomes não bater com a quantidade de valores?

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the text 'python'. The terminal contains two lines of Python code: 'a, b, c = 1, 2' and 'print(a, b, c)'.

```
python  
  
a, b, c = 1, 2  
  
print(a, b, c)
```

Erro em tempo de execução ou erro na hora de escrever o arquivo?

Quebra. E quebra só na hora de rodar, não antes.

python

```
a, b, c = 1, 2
```

SAÍDA

Traceback (most recent call last):

File "variaveis.py", line 1, in <module>

ValueError: not enough values to unpack (expected 3, got 2)

Guarde esse ponto: em Python o erro aparece quando a linha executa. Se a linha estiver dentro de um if que nunca roda, o defeito viaja para produção.

Trocar dois valores de lugar

Em Java ou C isso exige uma terceira variável temporária. Aqui não.

```
troca.py  
  
a, b = 10, 20  
  
a, b = b, a  
  
print(a, b)
```

SAÍDA

20 10

A mesma variável pode mudar de tipo no meio do programa?

```
python  
  
x = 5  
print(type(x))  
  
x = "cinco"  
print(type(x))
```

Pense em como isso seria em Java, com `int x = 5`.

Funciona. O tipo pertence ao valor, não ao nome.

python

```
x = 5
print(type(x))

x = "cinco"
print(type(x))
```

SAÍDA

```
<class 'int'>
<class 'str'>
```

Um nome em Python é uma etiqueta colada em um objeto. Reatribuir é descolar a etiqueta e colar em outro objeto, de outro tipo se for o caso.



A liberdade cobra o preço na hora errada. O erro de tipo em Python só aparece quando a linha executa.

É o argumento central de quem prefere linguagem de tipagem estática em sistema grande.

Palavras reservadas

Nenhuma delas pode virar nome de variável. São 35 no Python 3.12.

<code>False</code>	<code>None</code>	<code>True</code>	<code>and</code>	<code>as</code>
<code>assert</code>	<code>async</code>	<code>await</code>	<code>break</code>	<code>class</code>
<code>continue</code>	<code>def</code>	<code>del</code>	<code>elif</code>	<code>else</code>
<code>except</code>	<code>finally</code>	<code>for</code>	<code>from</code>	<code>global</code>
<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>
<code>return</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>

Para ver a lista na sua versão: `import keyword` e depois `print(keyword.kwlist)`

Usar palavra reservada como nome nem chega a rodar.

erro.py

```
class = "ADS 2026.2"
```

```
print(class)
```

SAÍDA

File "erro.py", line 1

class = "ADS 2026.2"

SyntaxError: invalid syntax

Erro de sintaxe é detectado antes de executar qualquer linha. Repare que nem o print da linha 3 chegou a acontecer.

Prática 1

8 min

Digite e rode. Cada linha imprime alguma coisa.

- 0 Crie curso, semestre e turno em uma única linha, com os valores "ADS", 2 e "noite".
- 1 Imprima os três separados por espaço.
- 2 Troque o valor de curso e turno de lugar sem usar variável auxiliar.
- 3 Imprima de novo e confira o resultado.
- 4 Atribua 0 a tres contadores diferentes em uma linha só.
- 5 Tente criar uma variável chamada for e anote a mensagem de erro.

Prática 1: solução

```
pratica1.py

curso, semestre, turno = "ADS", 2, "noite"
print(curso, semestre, turno)

curso, turno = turno, curso
print(curso, semestre, turno)

a = b = c = 0
print(a, b, c)
```

SAÍDA

```
ADS 2 noite
noite 2 ADS
0 0 0
```

02

Tipos e conversão

Onde a maior parte dos bugs de aluno iniciante nasce.

O usuário digita 21. Este programa imprime 22?

python

```
idade = input("Idade: ")
```

```
print(type(idade))
```

```
print(idade + 1)
```

A pergunta de verdade é: o que exatamente input devolve?

Não. input sempre devolve texto, mesmo quando o usuário digita número.

python

```
idade = input("Idade: ")

print(type(idade))
print(idade + 1)
```

SAÍDA

Idade: 21

<class 'str'>

TypeError: can only concatenate str (not "int") to str

O + em Python significa somar entre números e concatenar entre textos. Misturar os dois não tem significado definido, então a linguagem recusa.

A conversão precisa ser explícita

À esquerda o que quebra. À direita o que resolve.

```
errado.py

idade = input("Idade: ")

print(idade + 1)

# TypeError
```

```
certo.py

idade = int(input("Idade: "))

print(idade + 1)

# 22
```

Quais dessas três linhas quebram?

```
python  
  
print(int("3"))  
print(int(3.9))  
print(int("3.5"))
```

Duas passam. Uma não.

A terceira quebra. int não lê texto com ponto decimal.

python

```
print(int("3"))  
print(int(3.9))  
print(int("3.5"))
```

SAÍDA

3

3

ValueError: invalid literal for int() with base 10: '3.5'

int(3.9) trunca um número que já é número. int("3.5") pede para interpretar um texto que não é inteiro. Para esse caso use int(float("3.5")).

Conversões que você vai usar toda semana

Chamada	Entra	Sai	Cuidado
<code>int("42")</code>	texto	42	Espaço em branco nas pontas é tolerado
<code>int(3.9)</code>	float	3	Trunca, não arredonda
<code>float("3,5")</code>	texto	ValueError	Python usa ponto, não vírgula
<code>str(42)</code>	int	"42"	Sempre funciona
<code>bool("False")</code>	texto	True	Qualquer texto não vazio é verdadeiro
<code>round(3.9)</code>	float	4	Esse sim arredonda

Este teste passa?

```
python

print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

total = 0.0
for _ in range(10):
    total += 0.1

print(total)
```

Somar dez vezes dez centavos deveria dar um real.

Não passa. E isso não é bug do Python, é como o hardware guarda fração binária.

python

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(total)
```

SAÍDA

0.30000000000000004

False

0.9999999999999999

0.1 não tem representação exata em base 2, do mesmo jeito que 1/3 não tem em base 10. O padrão IEEE 754 vale para Java, C, JavaScript e praticamente todas as linguagens.

Dinheiro não se guarda em float

Decimal faz a conta em base 10, que é a base em que o cliente pensa.

```
dinheiro.py

from decimal import Decimal

print(Decimal("0.1") + Decimal("0.2"))
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

SAÍDA

0.3

True



**Repare que Decimal recebe texto, não número.
Decimal(0.1) já nasce com o erro do float embutido.**

Python Docs, Floating Point Arithmetic: Issues and Limitations

Prática 2

10 min

Programa de orçamento. Leia do teclado e imprima formatado.

- 0 Leia o preço unitário de um produto como float.
- 1 Leia a quantidade como int.
- 2 Calcule o total e aplique 10 por cento de desconto.
- 3 Imprima com duas casas decimais, usando f-string.
- 4 Teste com preço 19.90 e quantidade 3.
- 5 Imprima também `preco * qtd` sem formatação e compare os dois números.

Prática 2: solução

```
pratica2.py

preco = float(input("Preço unitário: "))
qtd = int(input("Quantidade: "))

total = preco * qtd
com_desconto = total * 0.9

print(total)
print(f"Total: R$ {com_desconto:.2f}")
```

SAÍDA

```
Preço unitário: 19.90
Quantidade: 3
59.699999999999996
Total: R$ 53.73
```



O 59.69999999999999996 estava lá o tempo todo. A f-string só escondeu, não corrigiu.

03

Condicionais

if, elif, else, e as duas armadilhas que aparecem em toda prova.

A estrutura básica

Sem chaves e sem parênteses. A indentação é a sintaxe.

```
nota.py

nota = float(input("Nota: "))

if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

SAÍDA

Nota: 6.5

Recuperação

Quais dessas linhas imprimem True?

```
python  
  
print(bool(0))  
print(bool(""))  
print(bool([]))  
print(bool("0"))  
print(bool(" "))  
print(bool("False"))
```

Três delas.

As três últimas. Qualquer texto não vazio é verdadeiro, inclusive "0" e "False".

```
python
print(bool(0))
print(bool(""))
print(bool([]))
print(bool("0"))
print(bool(" "))
print(bool("False"))
```

SAÍDA

False

False

False

True

True

True

Como input devolve texto, um if resposta: quase sempre é verdadeiro. É por isso que a comparação precisa ser explícita.

Tudo que Python considera falso

A lista é curta. Fora dela, é verdadeiro.

Valor	Tipo	bool()
<code>False</code>	<code>bool</code>	<code>False</code>
<code>None</code>	<code>NoneType</code>	<code>False</code>
<code>0</code> ou <code>0.0</code>	<code>int</code> , <code>float</code>	<code>False</code>
<code>""</code>	<code>str</code> vazia	<code>False</code>
<code>[]</code> <code>()</code> <code>{}</code>	coleção vazia	<code>False</code>
<code>"0"</code> <code>" "</code> <code>" "</code> <code>[0]</code>	não vazios	<code>True</code>

O usuário digitou n. Por que o programa continua?

```
python  
  
resposta = input("Continuar? (s/n) ")  
  
if resposta == "s" or "S":  
    print("continuando")
```

Leia a condição em voz alta, exatamente como está escrita.

Porque Python lê isso como (resposta == "s") or ("S"), e "S" é sempre verdadeiro.

python

```
resposta = "n"

print(resposta == "s" or "S")
print(resposta in ("s", "S"))
print(resposta.lower() == "s")
```

SAÍDA

True
False
False

or não distribui a comparação. Use in com uma tupla de opções, ou normalize o texto com lower antes de comparar.

Comparação encadeada

Escrever intervalo em Python é igual a escrever em matemática. Poucas linguagens permitem isso.

```
faixa.py

idade = 30

print(18 <= idade <= 65)
print(idade >= 18 and idade <= 65)
```

SAÍDA

True

True

Prática 3

12 min

Calculadora de frete de uma loja online.

- 0 Leia o valor da compra como float.
- 1 Leia se o cliente tem cupom, respondendo s ou n.
- 2 Com cupom, ou acima de R\$ 300, o frete é zero.
- 3 De R\$ 100 até R\$ 300, o frete é R\$ 8.
- 4 Abaixo de R\$ 100, o frete é R\$ 15.
- 5 Imprima o frete e o total, os dois com duas casas.

Prática 3: solução

```
pratica3.py

valor = float(input("Valor da compra: "))
cupom = input("Tem cupom? (s/n) ").lower() == "s"

if cupom or valor > 300:
    frete = 0.0
elif valor >= 100:
    frete = 8.0
else:
    frete = 15.0

print(f"Frete: R$ {frete:.2f}")
print(f"Total: R$ {valor + frete:.2f}")
```

SAÍDA

```
Valor da compra: 150
Tem cupom? (s/n) n
Frete: R$ 8.00
Total: R$ 158.00
```

04

Listas, tuplas e dicionários

As três estruturas que resolvem quase tudo no dia a dia.

As quatro coleções, lado a lado

Coleção	Cria com	Ordenada	Mutável	Aceita repetido
Lista	<code>[]</code>	sim	sim	sim
Tupla	<code>()</code>	sim	não	sim
Dicionário	<code>{ chave: valor }</code>	sim, desde 3.7	sim	valor sim, chave não
Conjunto	<code>{ }</code> ou <code>set()</code>	não	sim	não

Escolher errado aqui é a diferença entre um código que se explica sozinho e um que precisa de comentário.

Lista: acesso por índice

O primeiro item é o zero. O índice negativo conta de trás para frente.

```
listas.py

disciplinas = ["Coding", "Cloud", "Requisitos"]

print(disciplinas[0])
print(disciplinas[-1])
print(len(disciplinas))
```

SAÍDA

Coding

Requisitos

3

A lista tem três itens. O que sai em disciplinas[3]?

```
python  
  
disciplinas = ["Coding", "Cloud", "Requisitos"]  
  
print(disciplinas[3])
```

Nem sempre a resposta é um erro. Em C, por exemplo, não seria.

IndexError. Python confere o limite antes de acessar.

```
python  
  
disciplinas = ["Coding", "Cloud", "Requisitos"]  
  
print(disciplinas[3])
```

SAÍDA

```
Traceback (most recent call last):  
IndexError: list index out of range
```

Em C o mesmo acesso leria memória vizinha e devolveria lixo, sem avisar. Essa checagem custa alguns ciclos e evita uma classe inteira de falha de segurança.

Fatiamento

lista[inicio:fim], onde o fim não entra. O passo negativo inverte.

fatias.py

```
notas = [5, 7, 9, 6, 10]
```

```
print(notas[1:3])
```

```
print(notas[:2])
```

```
print(notas[2:])
```

```
print(notas[::-1])
```

SAÍDA

```
[7, 9]
```

```
[5, 7]
```

```
[9, 6, 10]
```

```
[10, 6, 9, 7, 5]
```


Métodos de lista

Todos alteram a lista no lugar, exceto copy e count.

Método	O que faz	Exemplo
append(x)	Acrescenta um item no fim	<code>a.append("Cloud")</code>
extend(lista)	Acrescenta cada item de outra sequência	<code>a.extend(["Cloud", "IA"])</code>
insert(i, x)	Insere na posição indicada	<code>a.insert(0, "Coding")</code>
remove(x)	Remove a primeira ocorrência do valor	<code>a.remove("Cloud")</code>
pop(i)	Remove e devolve o item da posição	<code>ultimo = a.pop()</code>
sort()	Ordena no lugar	<code>notas.sort()</code>
copy()	Devolve uma lista nova com os mesmos itens	<code>b = a.copy()</code>

append e extend recebem a mesma lista. O resultado é o mesmo?

```
python  
  
a = [1, 2]  
a.append([3, 4])  
print(a)
```

```
  
b = [1, 2]  
b.extend([3, 4])  
print(b)
```

Não. `append` coloca a lista inteira como um item só.

```
python  
  
a = [1, 2]  
a.append([3, 4])  
print(a)  
  
b = [1, 2]  
b.extend([3, 4])  
print(b)
```

SAÍDA

```
[1, 2, [3, 4]]  
[1, 2, 3, 4]
```

a ficou com 3 itens e b com 4. Um `len` no lugar errado depois disso gera relatório com número errado.

Ninguém tocou em turma_a. Por que ela mudou?

python

```
turma_a = ["Ana", "Bruno"]  
turma_b = turma_a  
  
turma_b.append("Carla")  
  
print(turma_a)
```

Esta é a pegadinha que mais aparece em código de aluno e também em código de gente experiente.

Porque turma_b não é uma cópia. São dois nomes colados no mesmo objeto.

```
python

turma_a = ["Ana", "Bruno"]
turma_b = turma_a

turma_b.append("Carla")

print(turma_a)
print(turma_a is turma_b)
```

SAÍDA

```
['Ana', 'Bruno', 'Carla']
True
```

O sinal de igual nunca copia o objeto, só copia a referência. is compara identidade e responde True: é literalmente a mesma lista na memória.

A correção

copy cria um objeto novo. A partir daí as duas listas seguem caminhos separados.

```
copia.py

turma_a = ["Ana", "Bruno"]
turma_b = turma_a.copy()

turma_b.append("Carla")

print(turma_a)
print(turma_b)
```

SAÍDA

```
['Ana', 'Bruno']
['Ana', 'Bruno', 'Carla']
```



Em Python, atribuir nunca copia. Guarde essa frase, ela explica metade dos bugs estranhos que você vai encontrar.

Ned Batchelder, Facts and Myths about Python names and values

Tupla: igual a lista, mas travada

Use quando o conjunto de valores não deve mudar, como coordenadas ou um registro fixo.

```
tuplas.py

ponto = (10, 20)

print(ponto[0])

ponto[0] = 99
```

SAÍDA

10

TypeError: 'tuple' object does not support item assignment

a e b têm o mesmo conteúdo. Têm o mesmo tipo?

```
python  
  
a = ("Coding")  
b = ("Coding",)  
  
print(type(a))  
print(type(b))
```

A diferença entre as duas linhas é um caractere.

Não. Quem cria a tupla é a vírgula, não os parênteses.

```
python  
  
a = ("Coding")  
b = ("Coding",)  
  
print(type(a))  
print(type(b))
```

SAÍDA

```
<class 'str'>  
<class 'tuple'>
```

Em a os parênteses são só agrupamento, como em uma conta. Uma função que devolve ("ok") devolve texto, e o for em cima disso vai iterar letra por letra.

Dicionário: acesso por chave

Quando o dado tem nome, dicionário é melhor que lista com índice mágico.

```
dicionarios.py

aluno = {"nome": "Ana", "matricula": 20261234, "media": 8.75}

print(aluno["nome"])
print(aluno.get("curso"))
print(aluno.get("curso", "ADS"))
```

SAÍDA

Ana

None

ADS

Qual das duas linhas derruba o programa?

python

```
aluno = {"nome": "Ana"}  
  
print(aluno.get("curso"))  
print(aluno["curso"])
```

A última. Colchete exige que a chave exista, get aceita a ausência.

python

```
aluno = {"nome": "Ana"}  
  
print(aluno.get("curso"))  
print(aluno["curso"])
```

SAÍDA

None

KeyError: 'curso'

Em dado que vem de fora, como formulário ou API, prefira get com valor padrão. Colchete é para chave que você tem certeza que existe.

Dicionário com lista dentro

A partir daqui já dá para modelar uma turma inteira.

```
turma.py

turma = {
    "Ana": [8.0, 9.5],
    "Bruno": [6.0, 5.5],
}

for nome, notas in turma.items():
    print(f"{nome}: {sum(notas) / len(notas):.1f}")
```

SAÍDA

Ana: 8.8

Bruno: 5.8

Conjunto: duplicidade some sozinha

Contar quantos alunos distintos assinaram a lista de presença.

```
presencas = ["Ana", "Bruno", "Ana", "Carla", "Bruno"]

print(len(presencas))
print(len(set(presencas)))
print(sorted(set(presencas)))
```

SAÍDA

5

3

['Ana', 'Bruno', 'Carla']

Prática 4

15 min

Diário de classe. Junta tudo que vimos hoje.

- 0 Monte um dicionário turma com Ana [8.0, 9.5], Bruno [6.0, 5.5] e Carla [4.0, 3.5].
- 1 Percorra o dicionário com items e calcule a média de cada aluno.
- 2 Classifique: 7 ou mais Aprovado, 5 ou mais Recuperação, abaixo disso Reprovado.
- 3 Imprima nome, média com uma casa e situação, alinhados.
- 4 Guarde as médias em uma lista e imprima a média da turma com duas casas.
- 5 Extra: use sorted para listar da maior média para a menor.

Prática 4: solução

```
pratica4.py

turma = {"Ana": [8.0, 9.5], "Bruno": [6.0, 5.5], "Carla": [4.0, 3.5]}

medias = []

for nome, notas in turma.items():
    media = sum(notas) / len(notas)
    medias.append(media)

    if media >= 7:
        situacao = "Aprovado"
    elif media >= 5:
        situacao = "Recuperação"
    else:
        situacao = "Reprovado"

    print(f"{nome:<8} {media:.1f} {situacao}")

print(f"Média da turma: {sum(medias) / len(medias):.2f}")
```

SAÍDA

Ana	8.8	Aprovado
Bruno	5.8	Recuperação
Carla	3.8	Reprovado
Média da turma: 6.08		

05

No domínio da sua equipe

O mesmo conteúdo, agora dentro do projeto que vocês escolheram na Aula 1.

Os domínios escolhidos

Cada equipe tem um catálogo, uma lista de itens consumidos e pelo menos uma regra chata.

Domínio	O que entra no catálogo	A regra chata
Barbearia	cutte, barba, sobancelha, hidratação	Desconto no combo a partir de três serviços
Petshop	banho, tosa, hidratação, transporte	Taxa extra para porte grande
Oficina	troca de óleo, alinhamento, revisão	Peça cobrada à parte da mão de obra
Biblioteca	empréstimo, renovação, reserva	Multa por dia de atraso
Estoque	entrada, saída, ajuste de inventário	Alerta abaixo do estoque mínimo
Escola de idiomas	matrícula, mensalidade, material	Desconto para pagamento até o dia 5

Se o domínio da sua equipe não está aqui, a estrutura é a mesma. Troque só os nomes.

Prática 5: comanda do seu domínio

20 min

Use o domínio da sua equipe. Quem estiver sem equipe hoje, use barbearia.

- 0 Monte um dicionário `precos` com quatro itens do seu catálogo, os valores em `Decimal`.
- 1 Monte uma lista `comanda` com o que o cliente consumiu, repetindo pelo menos um item.
- 2 Inclua de propósito um item que não existe no catálogo.
- 3 Percorra a `comanda` somando. Item fora da tabela: avise na tela e siga com `continue`.
- 4 Aplique a regra chata da sua equipe. Na barbearia, 10 por cento a partir de três itens.
- 5 Imprima quantidade, itens distintos, subtotal, desconto e total, todos formatados.

Prática 5: solução, parte 1

O catálogo é um dicionário. A comanda é uma lista. A soma é um laço com uma guarda.

```
pratica5_comanda.py

from decimal import Decimal

precos = {
    "corte": Decimal("35.00"),
    "barba": Decimal("25.00"),
    "sobrancelha": Decimal("15.00"),
    "hidratação": Decimal("40.00"),
}

comanda = ["corte", "barba", "corte", "pintura"]

total = Decimal("0.00")

for item in comanda:
    if item not in precos:
        print(f"item fora da tabela: {item}")
        continue
    total += precos[item]
```

Prática 5: solução, parte 2

A regra chata e a impressão do fechamento.

```
pratica5_comanda.py

if len(comanda) >= 3:
    desconto = total * Decimal("0.10")
else:
    desconto = Decimal("0.00")

print(f"Itens:      {len(comanda)}")
print(f"Distintos: {sorted(set(comanda))}")
print(f"Subtotal:   R$ {total:.2f}")
print(f"Desconto:    R$ {desconto:.2f}")
print(f"Total:      R$ {total - desconto:.2f}")
```

SAÍDA

```
item fora da tabela: pintura
Itens:      4
Distintos:  ['barba', 'corte', 'pintura']
Subtotal:   R$ 95.00
Desconto:    R$ 9.50
Total:      R$ 85.50
```



**A conta cobrou três serviços e a tela informou quatro itens.
O programa roda, não dá erro, e o cliente reclama no caixa.**

Defeito de regra de negócio não aparece em traceback. É esse tipo de falha que Engenharia de Requisitos existe para prevenir.

Como corrigir a contagem sem reescrever o programa inteiro?

```
python

validos = []

for item in comanda:
    if item not in precos:
        print(f"item fora da tabela: {item}")
        continue
    validos.append(item)
    total += precos[item]
```

Depois disso, `len(validos)` e `set(validos)` passam a contar o que foi de fato cobrado.

06

Fechar a aula no Git

Aula que não chega ao repositório não conta como entregue.

Passo 1: organizar os arquivos

Cinco arquivos, um por prática, dentro de uma pasta com o número da aula.

terminal

```
cd ads-2026-2-barbearia
```

```
mkdir aula03
```

```
# mova ou salve os cinco arquivos aqui dentro:
```

```
# aula03/pratica1_variaveis.py
```

```
# aula03/pratica2_orcamento.py
```

```
# aula03/pratica3_frete.py
```

```
# aula03/pratica4_diario.py
```

```
# aula03/pratica5_comanda.py
```

Passo 2: ver o que mudou antes de adicionar

Este comando é o mais importante do dia. Nunca adicione o que você não olhou.

 terminal

```
git status
```

SAÍDA

On branch main

Untracked files:

(use "git add <file>..." to include in what will be committed)

aula03/

nothing added to commit but untracked files present

Passo 3: git add, arquivo por arquivo

Nomear cada arquivo é mais lento e evita subir o que não devia.

terminal

```
git add aula03/pratica1_variaveis.py
git add aula03/pratica2_orcamento.py
git add aula03/pratica3_frete.py
git add aula03/pratica4_diario.py
git add aula03/pratica5_comanda.py
```

atalho equivalente, quando a pasta só tem o que você quer:

git add aula03/

Evite git add ponto. Ele leva junto tudo que estiver por perto.

```
terminal

git add .

# nesta pasta isso levaria tambem:
# .env com senha do banco
# __pycache__/ com arquivos compilados
# venv/ com o ambiente inteiro
# captura_de_tela.png de 4 MB
```

SAÍDA

1247 arquivos
adicionados

Chave que entra no histórico do Git continua lá mesmo depois de apagada em um commit seguinte. Um arquivo .gitignore resolve isso de forma definitiva.

Passo 4: conferir o que está na área de preparo

Rode git status de novo. Agora os arquivos aparecem em verde.

A terminal window with a dark background. The title bar shows three colored circles (red, yellow, green) and the word 'terminal'. The command 'git status' is entered in a light blue monospace font.

```
git status
```

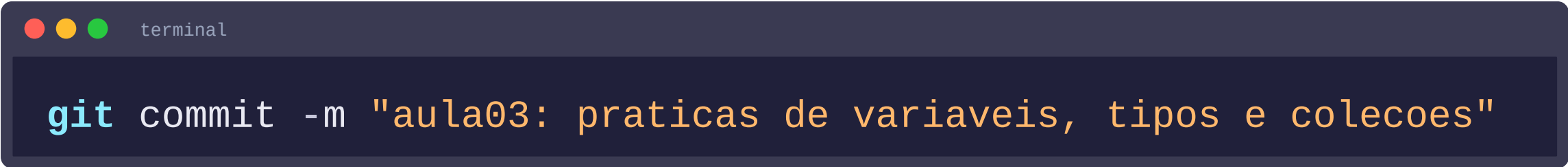
SAÍDA

Changes to be committed:

```
new file:   aula03/pratica1_variaveis.py
new file:   aula03/pratica2_orcamento.py
new file:   aula03/pratica3_frete.py
new file:   aula03/pratica4_diario.py
new file:   aula03/pratica5_comanda.py
```

Passo 5: a frase do commit

Escreva no imperativo, dizendo o que o commit faz com o projeto.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'terminal' is visible in the title bar. The command 'git commit -m "aula03: praticas de variaveis, tipos e colecoes"' is entered in a monospaced font, with 'git' in blue and the rest in orange.

```
git commit -m "aula03: praticas de variaveis, tipos e colecoes"
```

SAÍDA

```
[main 7f3a1c2] aula03: praticas de variaveis, tipos e colecoes  
5 files changed, 118 insertions(+)
```

Mensagem de commit que presta

A pergunta que a frase responde: se este commit for aplicado, o que acontece com o projeto?

Não faça	Faça
"alteracoes"	"aula03: adiciona praticas de colecoes"
"aula 3"	"comanda: aplica desconto a partir de 3 itens"
"funcionou agora vai"	"corrige contagem de itens da comanda"
"asdasd"	"remove venv do controle de versao"
"atualizacao final v2 FINAL"	"README: descreve o dominio da equipe"

Padrão sugerido: prefixo curto, dois pontos, verbo no presente. Sem ponto final.

Passo 6: enviar para o GitHub

Até aqui tudo aconteceu só na sua máquina. O push é o que torna a entrega visível.

 terminal

```
git push origin main
```

SAÍDA

```
Enumerating objects: 9, done.
```

```
To https://github.com/equipe/ads-2026-2-barbearia.git
```

```
a1b2c3d..7f3a1c2  main -> main
```

Push recusado significa que um colega enviou antes de você.

terminal

```
git push origin main
```

a correção:

```
git pull --rebase origin main
```

```
git push origin main
```

SAÍDA

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs
```

O rebase reaplica os seus commits em cima do que já estava no servidor, sem criar commit de merge. Se houver conflito, o Git para e aponta o arquivo.

Antes de sair da sala

checklist

- 0 Os cinco arquivos rodam sem erro, cada um no seu terminal.
- 1 `git status` responde: `nothing to commit, working tree clean`.
- 2 A página do repositório no GitHub mostra a pasta `aula03`.
- 3 O nome de cada integrante aparece em pelo menos um commit da semana.
- 4 Nenhum arquivo de senha, ambiente virtual ou `__pycache__` foi enviado.

O que ficou de hoje

Ponto	Regra prática
Tipo	Pertence ao valor, não ao nome. O erro só aparece ao executar
input	Devolve texto sempre. Converta antes de calcular
Dinheiro	Decimal com texto na entrada, nunca float
Condição	Compare explicitamente. or não distribui
Atribuição	Nunca copia. Use copy quando quiser objeto novo
Tupla	Quem cria é a vírgula
Dicionário	get para dado externo, colchete para chave garantida

Referências

Python Docs, Tutorial oficial em português

<https://docs.python.org/pt-br/3/tutorial/introduction.html>

Python Docs, Estruturas de dados

<https://docs.python.org/pt-br/3/tutorial/datastructures.html>

Python Docs, Aritmética de ponto flutuante e suas limitações

<https://docs.python.org/pt-br/3/tutorial/floatingpoint.html>

Ned Batchelder, Facts and Myths about Python names and values

<https://nedbatchelder.com/text/names.html>

Real Python, Lists and Tuples in Python

<https://realpython.com/python-lists-tuples/>

Real Python, Dictionaries in Python

<https://realpython.com/python-dicts/>

PEP 8, guia de estilo para código Python

<https://peps.python.org/pep-0008/>

RAMALHO, Luciano. Python Fluente. 2. ed. Novatec, 2023. Capítulos 2 e 6

<https://novatec.com.br/livros/python-fluente-2ed/>

CHACON e STRAUB. Pro Git. 2. ed. Apress, 2014. Capítulo 2, em português

<https://git-scm.com/book/pt-br/v2>

Conventional Commits, convenção de mensagem de commit

<https://www.conventionalcommits.org/pt-br/v1.0.0/>



Rode tudo de novo em casa. Código que você só leu não conta.

Entrega: pasta aula03 com as cinco práticas no repositório da equipe, com push feito, até domingo às 23h59.

Próxima aula: Laços, funções e a primeira função com teste automatizado em pytest.